

JOINS e Relacionamentos

Tipos de JOIN (INNER, LEFT e RIGHT) · Relacionamentos em bancos de dados · Atividade prática com tabelas associativas

BANCO DE DADOS

SQL



Objetivos de Aprendizagem

1

Tipos de JOIN

Compreender os tipos de JOIN e suas aplicações práticas em consultas SQL.

2

Relacionamentos

Trabalhar com relacionamentos entre tabelas: 1-1, 1-N e N-N.

3

Tabelas Associativas

Utilizar JOIN em tabelas associativas para relacionamentos muitos para muitos.

4

Múltiplas Tabelas

Realizar consultas envolvendo três ou mais tabelas simultaneamente.

O que é JOIN?

Definição

JOIN é usado para combinar dados de duas ou mais tabelas com base em uma condição de relacionamento.

Como Funciona

Ele permite unir registros que têm valores correspondentes em uma coluna específica, tornando possível consultar informações distribuídas em diferentes tabelas como se fossem uma só.

-  O JOIN é um dos recursos mais poderosos do SQL para análise relacional de dados.



Principais Tipos de JOIN

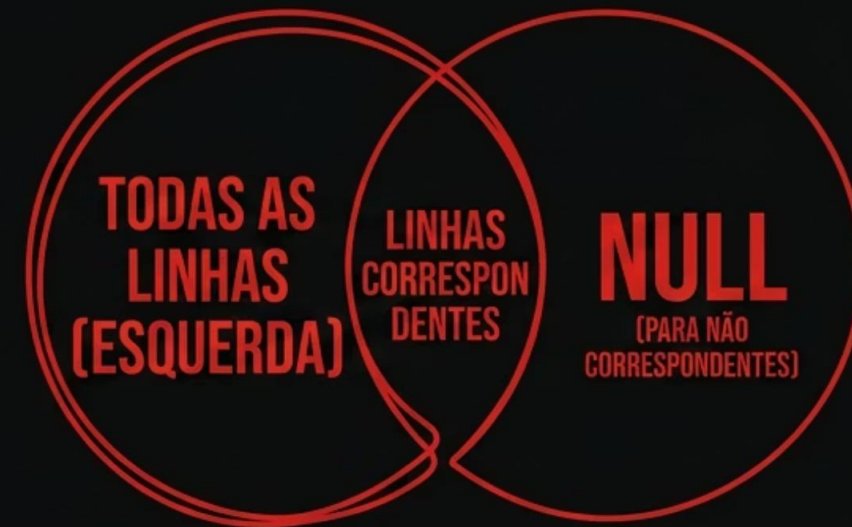
INNER JOIN

Retorna **apenas as linhas** que possuem correspondência em **ambas** as tabelas.



LEFT JOIN

Retorna **todas as linhas** da tabela da esquerda e as correspondências da direita. Sem correspondência: **NULL**.



INNER JOIN – Como Funciona

- ✓ **Retorna:** Apenas as linhas em que há correspondência em ambas as tabelas.

Cenário de Uso

Relacionamento entre alunos e cursos onde **somente os alunos matriculados** em cursos são retornados. Registros sem correspondência são excluídos do resultado.

Exemplo SQL

```
SELECT alunos.nome, cursos.nome
FROM alunos
INNER JOIN matriculas
    ON alunos.id = matriculas.aluno_id
INNER JOIN cursos
    ON cursos.id = matriculas.curso_id;
```

Nesse caso, a função do **INNER JOIN** é funcionar como um **filtro de correspondência obrigatória** e uma **ponte de ligação**.

Aqui estão as duas funções práticas que ele está exercendo:

1. Fazer a "Ponte" entre duas tabelas que não se conhecem

As tabelas alunos e cursos não têm nenhuma coluna em comum. Se você tentasse ligar as duas diretamente, o banco de dados não saberia quem pertence a quem.

O **INNER JOIN** resolve isso usando a tabela matriculas como uma ponte de duas etapas:

- O primeiro **INNER JOIN** amarra o Aluno à sua Matrícula.
- O segundo **INNER JOIN** pega essa Matrícula e amarra ao seu Curso correspondente.

LEFT JOIN – Como Funciona

 **Retorna:** Todas as linhas da tabela da esquerda e as correspondências da tabela da direita. Se não houver correspondência, valores **NULL** são retornados.

Cenário de Uso

Retornar **todos os alunos**, inclusive aqueles que não estão matriculados em nenhum curso. Os campos do curso aparecerão como NULL para esses alunos.

Exemplo SQL

```
SELECT alunos.nome, cursos.nome  
FROM alunos  
LEFT JOIN matriculas  
  ON alunos.id = matriculas.aluno_id  
LEFT JOIN cursos  
  ON cursos.id = matriculas.curso_id;
```

Mudou apenas uma palavra na sua consulta (INNER virou LEFT), mas o resultado que ela traz agora é **completamente diferente**.

Essa nova consulta traz uma **lista de TODOS os alunos cadastrados no banco, sem exceção**. Se o aluno estiver matriculado em um curso, o nome do curso aparece; se ele não estiver matriculado em nada, o nome dele continua aparecendo na lista, mas o campo do curso fica vazio (NULL).

Em termos simples, ela responde à pergunta: *"Quem são todos os nossos alunos e quais cursos eles estão fazendo (se é que estão fazendo algum)?"*

RIGHT JOIN – Como Funciona

⚠ **Retorna:** Todas as linhas da tabela da direita e as correspondências da tabela da esquerda. Se não houver correspondência, valores **NULL** são retornados.

Cenário de Uso

Retornar **todos os cursos**, inclusive aqueles sem alunos matriculados. Os campos do aluno aparecerão como NULL para esses cursos.

Exemplo SQL

```
SELECT alunos.nome, cursos.nome  
FROM alunos  
RIGHT JOIN matriculas  
    ON alunos.id = matriculas.aluno_id  
RIGHT JOIN cursos  
    ON cursos.id = matriculas.curso_id;
```

Se no LEFT JOIN o foco era total nos alunos, agora o jogo virou completamente! Ao mudar para **RIGHT JOIN**, o banco de dados mudou o lado da prioridade.

Esta consulta traz uma **lista de TODOS os cursos cadastrados no banco, sem exceção**. Se o curso tiver alunos matriculados, o nome do aluno aparece ao lado; se for um curso novo ou que está sem nenhum aluno no momento, o nome do curso continua aparecendo na lista, mas o campo do aluno fica vazio (NULL).

Em termos simples, ela responde à pergunta: *"Quais são todos os cursos que a nossa escola oferece e quem está matriculado neles?"*

Relacionamentos em Bancos de Dados



1-1 (Um para Um)

Um registro em uma tabela está associado a **um único registro** em outra tabela.



1-N (Um para Muitos)

Um registro em uma tabela está associado a **vários registros** em outra tabela.



N-N (Muitos para Muitos)

Vários registros em uma tabela estão associados a vários registros em outra. **Requer tabela intermediária.**

Exemplos de Relacionamentos

Relacionamento 1-1: Pessoas e Documentos

Tabelas: `peessoas` e `documentos_identidade` — Cada pessoa tem um único documento de identidade.

```
SELECT pessoas.nome, documentos_identidade.numero
      FROM pessoas
      INNER JOIN documentos_identidade
    ON pessoas.id = documentos_identidade.pessoa_id;
```

Relacionamento 1-N: Clientes e Pedidos

Tabelas: `clientes` e `pedidos` — Um cliente pode ter vários pedidos, mas cada pedido pertence a um único cliente.

```
SELECT clientes.nome, pedidos.id
      FROM clientes
      INNER JOIN pedidos
    ON clientes.id = pedidos.cliente_id;
```

Relacionamento N-N: Estudantes e Disciplinas

Tabelas: `estudantes`, `disciplinas` e `matriculas` — Vários estudantes podem se matricular em várias disciplinas. A tabela `matriculas` é a tabela associativa.

```
SELECT estudantes.nome, disciplinas.nome
      FROM estudantes
      INNER JOIN matriculas
    ON estudantes.id = matriculas.estudante_id
      INNER JOIN disciplinas
    ON disciplinas.id = matriculas.disciplina_id;
```

Tabelas Associativas em N-N

Função

Uma **tabela associativa** é usada para armazenar os relacionamentos entre duas tabelas em um relacionamento N-N. Ela contém as chaves estrangeiras de ambas as tabelas relacionadas.

- ❏ A tabela `matriculas` armazena os IDs de estudantes e disciplinas, criando o vínculo entre elas.





Atividade Prática

Realize consultas envolvendo múltiplas tabelas, aplicando JOINS para validar o modelo entidade-relacionamento.

1 **INNER JOIN com Três Tabelas**

Crie uma consulta utilizando **INNER JOIN** para obter informações de alunos, cursos e professores simultaneamente.

2 **LEFT JOIN para Cursos sem Alunos**

Utilize **LEFT JOIN** para listar todos os cursos, incluindo os que não possuem alunos matriculados.

3 **RIGHT JOIN para Alunos sem Matrícula**

Faça uma consulta com **RIGHT JOIN** para mostrar todos os alunos, incluindo os que não estão matriculados em cursos.

Resumo Final



JOINs

Permitem combinar dados de diferentes tabelas, facilitando a análise de relacionamentos complexos em SQL.



Tipos de JOIN

INNER, LEFT e RIGHT — cada um adequado para diferentes cenários de consulta e necessidades de negócio.



Relacionamentos

1-1, 1-N e N-N, com tabelas associativas para implementar relacionamentos muitos para muitos.

Dominar JOINs e relacionamentos é essencial para construir consultas SQL eficientes e modelos de dados bem estruturados.